



GEBZE TECHNICAL UNIVERSITY
Department of Computer Engineering
CSE 396 - Computer Engineering Project | Spring 2026

Module Documentation

AI-Powered Smart PPE Inspection Station

GROUP-11



Team Members

Student Name	Student ID	Assigned Module
Muhammed Emir Kara	210104004071	Module 2 (Turnstile) — Primary
Mümincan Durak	210104004057	Module 2 (Turnstile) — Primary
Hüseyin Elyesa Yeşilyurt	210104004080	Module 4 (Backend) — Primary
Alperen Söylen	220104004024	Module 3 (IoT Module) — Primary
Ahmet Emre Kurt	220104004016	Module 4 (Backend) — Primary
Zeynep Etik	220104004035	Module 1 (AI & Vision) — Primary
Emre İlhan Şenel	230104004907	Module 5 (UI & UX) — Primary
Tarık Saeede	200104004804	Module 5 (UI & UX) — Primary

Team Members.....	2
1. MOD-01 AI & Vision Module.....	4
1.1 Module Overview.....	4
1.2 Background and Related Work.....	4
1.3 Methodology.....	4
1.4 Inter-Module Communication.....	5
1.5 Implementation.....	5
1.6 Testing Strategy.....	6
1.7 Known Limitations and Open Issues.....	7
2. MOD-02 Turnstile Module.....	8
2.1 Module Overview.....	8
2.2 Background and Related Work.....	8
2.3 Methodology.....	8
2.4 Inter-Module Communication.....	9
2.5 Implementation.....	9
2.6 Testing Strategy.....	10
2.7 Known Limitations and Open Issues.....	11
3. MOD-03 IoT Module.....	12
3.1 Module Overview.....	12
3.2 Background and Related Work.....	12
3.3 Methodology.....	12
3.4 Inter-Module Communication.....	13
3.5 Implementation.....	13
3.6 Testing Strategy.....	14
3.7 Known Limitations and Open Issues.....	14
4. MOD-04 Backend & Database Module.....	15
4.1 Module Overview.....	15
4.2 Background and Related Work.....	15
4.3 Methodology.....	15
4.4 Inter-Module Communication.....	16
4.5 Implementation.....	16
4.6 Testing Strategy.....	16
4.7 Known Limitations and Open Issues.....	24
5. MOD-05 UI & UX Module.....	25
5.1 Module Overview.....	25
5.A Admin Panel.....	25
5.A.1 Background and Related Work.....	25
5.A.2 Methodology.....	25
5.A.3 Inter-Module Communication.....	26
5.A.4 Implementation.....	26
5.A.5 Testing Strategy.....	26
5.A.6 Known Limitations and Open Issues.....	28
5.B Turnstile Display.....	28
5.B.1 Background and Related Work.....	28
5.B.2 Methodology.....	28
5.B.3 Inter-Module Communication.....	29

5.B.4 Implementation.....	29
5.B.5 Testing Strategy.....	30
5.B.6 Known Limitations and Open Issues.....	31

1. MOD-01 AI & Vision Module

1.1 Module Overview

The AI and Vision module is responsible for detecting personal protective equipment on camera frames captured by the IoT module and returning a list of identified PPE items to the caller. The module runs on a Raspberry Pi 5 equipped with a Hailo-8L AI Kit, which provides a dedicated neural processing unit for hardware-accelerated inference, and is implemented in Python 3.11. It receives a raw camera frame as a NumPy array from MOD-03 (IoT module), runs inference using a YOLOv8n model fine-tuned on a PPE dataset, and returns a list of detected item keys as strings to MOD-03 for the access decision. The module does not perform the access decision itself; that responsibility belongs to MOD-03, which compares the detected items against the worker's required PPE list retrieved from MOD-04 (Backend). The module exposes a single public function, `detect_ppe(frame: np.ndarray) -> list[str]`, which is the only interface MOD-03 depends on. Model training is performed separately on a development PC using the Ultralytics library, and the trained model is exported to ONNX format for deployment on the Raspberry Pi.

1.2 Background and Related Work

The dataset selection process began with the Ultralytics Construction PPE Dataset, which provides labeled images covering the core PPE classes required by the project: helmet, safety vest, gloves, safety boots, and goggles, along with their negative counterparts such as no-helmet and no-vest. This dataset was selected as the primary source because its class structure directly matched the inspection requirements of the system without requiring relabeling. However, the dataset alone proved insufficient for reliable detection of negative classes such as no-gloves and no-boots, which had significantly fewer labeled instances than the positive classes. To address this imbalance, a second dataset, "PPE Detection Computer Vision Model" by testcasque on Roboflow Universe, containing approximately 5,100 images with the same 10-class structure, was merged into the training set to increase sample diversity for the underrepresented classes.

The model architecture selection was guided by the accuracy-versus-latency tradeoff published on the Ultralytics website, which plots COCO mAP50-95 against parameter count and inference latency for all YOLOv8 variants. The nano variant, YOLOv8n, sits at the lowest end of both axes, meaning it has the fewest parameters and the shortest inference latency while still achieving acceptable detection accuracy. For a real-time inspection system running on embedded hardware, this tradeoff is more important than maximizing accuracy, because a model that misses the latency budget makes the system unusable regardless of its accuracy score. The larger variants, YOLOv8s and above, were considered but rejected because their parameter counts increase inference time beyond what the inspection cycle allows.

The trained model is exported to ONNX format for deployment because ONNX provides a hardware-agnostic intermediate representation that separates the training environment from the inference environment. This allows the model to be trained using the Ultralytics Python library on a development machine and then deployed on the Raspberry Pi without carrying the Ultralytics dependency into the production environment. The Hailo-8L NPU, which is the target inference accelerator on the Raspberry Pi 5 AI Kit, also uses ONNX as its input format for model compilation, making this choice a requirement of the hardware platform rather than an optional optimization.

1.3 Methodology

The AI and Vision module is organized into two distinct components: a training pipeline and an inference interface. The training pipeline runs exclusively on a development machine and is responsible for producing a trained model file from a labeled dataset. The inference interface runs on the Raspberry Pi 5 and is responsible for loading the trained model and executing detection on incoming camera frames. This separation exists because training a neural network requires GPU

resources that are not available on the Raspberry Pi, while inference must run on the target hardware under real-time constraints. The inference interface is implemented as a single class, `AIVisionImpl`, which inherits from the abstract `AIVisionModule` base class defined in `module_ai_vision.py`. This base class defines the public contract that MOD-03 depends on, ensuring that the implementation can be swapped or updated without affecting the caller.

Data enters the module as a NumPy array representing a 640x640 RGB camera frame, which MOD-03 is responsible for providing in the correct dimensions. Inside `detect_ppe()`, the frame is normalized to the range 0.0 to 1.0 by dividing by 255, transposed from HWC to CHW layout to match the ONNX model's expected input format, and expanded with a batch dimension before being passed to the ONNX Runtime inference session. The model produces a set of raw detections, each containing a bounding box, a confidence score, and a class index. The `_parse_outputs` method filters out detections below the confidence threshold, maps the class index to the corresponding `PPEClass` enum value, and returns the surviving detections as a list of item key strings. This list is the module's primary output and is passed directly to MOD-03 for the access decision.

The decision to use ONNX Runtime as the inference backend, rather than running the Ultralytics library directly on the Raspberry Pi, was made to minimize the dependency footprint on the production device. The Ultralytics library is large and includes training dependencies that are unnecessary at inference time. ONNX Runtime, by contrast, is a lightweight inference engine that loads and executes any ONNX-format model without requiring the original training framework. The Hailo-8L NPU, which is the intended hardware accelerator on the Raspberry Pi 5 AI Kit, requires a compiled model format specific to the Hailo toolchain. Because Hailo NPU integration is not yet complete, the module currently falls back to CPU-based ONNX Runtime inference. The `use_hailo_npu` flag in `AIVisionConfig` is reserved for this transition and has no effect in the current implementation.

The confidence threshold is set to 0.50 in the current implementation, with a target threshold of 0.75 for the production deployment. The lower value is used during development because the model has not yet been validated on the target hardware, and a stricter threshold may suppress valid detections whose confidence scores are compressed by the domain gap between the training dataset and real indoor lighting conditions. Once the module is tested on the Raspberry Pi with representative frames from the actual camera, the threshold will be adjusted based on the observed false positive and false negative rates.

The module is activated by MOD-03 during the INSPECTING state of the system state machine, after a worker's RFID card has been successfully read. Once `detect_ppe()` returns its result, MOD-03 takes over and makes the access decision based on the detected items.

1.4 Inter-Module Communication

The AI and Vision module communicates exclusively with MOD-03 (IoT module). This boundary exists because MOD-03 is the system orchestrator: it owns the camera pipeline, manages the inspection state machine, and is the only module that calls `detect_ppe()`. The AI and Vision module has no knowledge of which worker is being inspected, which PPE items are required for that worker's role, or what access decision follows from the detection result. All of that context lives in MOD-03, which is the appropriate place for it because MOD-03 is the module that coordinates all hardware inputs and outputs during the INSPECTING state. If this communication did not exist, there would be no mechanism to trigger inference or to deliver detection results to the part of the system that acts on them.

The direction of dependency is strictly one-way: MOD-03 calls into MOD-01, and MOD-01 returns a result. MOD-01 does not call back into MOD-03, does not hold state between frames, and does not initiate any communication on its own. This design ensures that the AI and Vision module remains stateless from the perspective of the inspection pipeline, which simplifies testing and allows the module to be developed and validated independently of the rest of the system. One unresolved constraint on this communication is the frame format: MOD-03 must provide a 640x640 square RGB frame, but the RPi Camera Module V3 does not output this resolution natively, and it has not yet

been confirmed which module is responsible for the crop and resize operation. This is identified as compatibility issue C-11 and remains open at the time of writing.

1.5 Implementation

The dataset preparation step presents the first non-trivial implementation challenge. The two source datasets use different class index schemes, so direct merging without remapping would cause the model to learn contradictory annotations. The `merge_datasets.py` script resolves this by defining an explicit index mapping from the testcasque dataset's class indices to the unified scheme and applying it to every label file during the copy process. The merged dataset contains 12 classes and is split into train, validation, and test subsets at an 80/10/10 ratio using a fixed random seed of 42 to ensure reproducibility. All image-label pairs from both sources are pooled and shuffled before splitting, so the split boundaries are independent of the source dataset.

The training run produces a model with strongly uneven per-class performance, which reflects the class imbalance in the merged dataset. Positive classes with high instance counts achieve reliable detection: helmet reaches a mAP50 of 0.945, vest reaches 0.963, and boots reaches 0.935. Negative classes, which have far fewer labeled instances, perform significantly worse: no_gloves reaches only 0.043, no_boots reaches 0.090, and no_helmet reaches 0.304. These negative classes are the safety-critical ones, because a missed negative detection means a worker without the required equipment is incorrectly granted access. The overall mAP50 across all 12 classes is 0.552, measured on the validation set after 50 epochs of training. Inference speed on the development machine is 4.4 milliseconds per frame, though this figure does not reflect performance on the Raspberry Pi CPU or the Hailo-8L NPU, neither of which has been benchmarked yet.

The output parsing logic in `_parse_outputs` assumes that the ONNX model returns detections in the format `[x_center, y_center, width, height, confidence, class_id]` per row. This assumption matches the standard YOLOv8 ONNX export format documented by Ultralytics, but it has not yet been verified by running inference on the Raspberry Pi. If the actual output tensor shape or field ordering differs, the parsing logic will produce incorrect results silently, because there is no shape validation before the loop. This is a known risk that will be addressed during hardware integration testing.

1.6 Testing Strategy

Testing for the AI and Vision module is conducted in two phases. The first phase covers model validation and is performed on the development machine using the Ultralytics validation pipeline, which evaluates per-class mAP50 scores on the held-out test split of the merged dataset. This phase is already complete for the current 50-epoch training run. The second phase covers inference integration and will be performed by running `ai_vision.py` against static test images, without requiring a physical camera or Raspberry Pi, to verify that the ONNX Runtime session loads correctly, that `_parse_outputs` produces the expected `DetectedPpe` list, and that the confidence threshold filters detections as intended. Hardware integration testing on the Raspberry Pi, including inference latency measurement and Hailo NPU validation, is planned but has not yet been conducted. The acceptance criterion for this module is a mAP50 score of at least 0.75 across all safety-critical PPE classes, particularly the negative classes such as no_helmet, no_vest, and no_gloves, which currently fall significantly below this threshold.

Test ID	Scenario	Input	Expected Output	Pass Criteria
T-01	Model loads successfully	Valid best.onnx path in AIVisionConfig	init() returns True	No exception, session is not None
T-02	Model load fails on invalid path	Non-existent model path	init() returns False	Exception caught, returns False without crash

T-03	Helmet detected on static image	640x640 RGB image containing a worker with helmet	detected_items contains item_key="helmet" with confidence above threshold	Correct class returned
T-04	Missing helmet detected	640x640 RGB image of worker without helmet	detected_items contains item_key="no_helmet"	Correct negative class returned
T-05	Low confidence detection filtered	Image with partially visible PPE item	Detection below 0.75 confidence not in result	Result list does not contain low-confidence item
T-06	Empty frame handling	Blank 640x640 black image	DetectionResult returned with success=True, empty detected_items	No crash, empty list returned
T-07	Multiple PPE classes detected	Image containing helmet, vest, and gloves	All three item keys present in detected_items	All classes correctly identified

1.7 Known Limitations and Open Issues

The most significant limitation of the current implementation is the low detection accuracy of the negative PPE classes, which are the classes that determine whether a worker is denied access. After 50 epochs of training on the merged dataset, no_gloves achieves a mAP50 of only 0.043, no_boots achieves 0.090, and no_helmet achieves 0.304, all of which fall far below the acceptance threshold of 0.75. This weakness is a direct consequence of class imbalance in the training data: no_boots has only 28 labeled instances in the validation set compared to 763 for helmet. Until additional labeled data is collected for these underrepresented classes and the model is retrained, the system cannot reliably detect missing PPE and may grant access to workers who are not properly equipped. This corresponds to risk R-03 in the project risk register. Additionally, Hailo-8L NPU integration is not yet implemented; the module currently runs inference on the Raspberry Pi CPU using ONNX Runtime, and inference latency on the target hardware has not been measured. Whether the system meets its real-time inspection requirement depends on this benchmark, which corresponds to risk R-02.

Two open issues remain unresolved at the time of writing. First, the PPE item key naming convention is not yet agreed upon across modules. The AI and Vision module currently returns class names derived from its training dataset, such as "helmet" and "no_vest", while MOD-04 and MOD-05 use underscore-prefixed strings such as "hard_hat" and "safety_vest" with no mapping defined between them. Without a conversion layer in MOD-03, every PPE comparison across the inspection pipeline will fail silently, as identified in compatibility note C-01. This issue affects MOD-01, MOD-03, MOD-04, and MOD-05, and the decision on the canonical naming convention must be made jointly by all teams before integration testing begins. Second, the frame preprocessing responsibility between MOD-01 and MOD-03 is not yet finalized. MOD-01 requires a 640x640 square RGB frame, but the RPi Camera Module V3 outputs a 4:3 or 16:9 aspect ratio by default. Compatibility note C-11 assigns the crop and resize responsibility to MOD-03, but this has not been confirmed by both teams, and no implementation exists yet on either side.

2. MOD-02 Turnstile Module

2.1 Module Overview

The Turnstile Module is the physical access control component of the inspection station, responsible for mechanically permitting or denying a worker's entry based on the outcome of the PPE inspection cycle. The turnstile is designed in CAD software and fabricated from wood, 3D-printed parts, and transparent materials such as acrylic or polycarbonate, mounted on a structural chassis that houses all electromechanical components. Two high-torque servo motors, connected to the Raspberry Pi 5 GPIO pins, drive the lock and rotation of the gate arm through PWM signals generated by the main controller. The module exposes gate control functions that are called in-process by the IoT Module (MOD-03) upon receiving an access decision. Within the overall inspection flow, the Turnstile Module is the last actor to execute: the gate remains locked throughout the IDENTIFYING and INSPECTING states and physically opens only when MOD-03 signals a GRANTED outcome. After a fixed passage timeout, the gate re-locks automatically and the system returns to the IDLE state.

2.2 Background and Related Work

Commercial pedestrian turnstile systems used in industrial and transit environments are typically fabricated from heavy-gauge stainless steel or carbon steel, with gate barriers made from tempered glass. These materials are chosen for their durability and resistance to mechanical wear over years of high-traffic use. However, sourcing, welding, and finishing these materials requires manufacturing lead times of several weeks and tooling costs that are incompatible with a university prototype built within a constrained development schedule. This gap between commercial manufacturing standards and rapid prototyping constraints is well established in engineering literature, and it directly motivated the decision to build the structural frame from wood, use 3D-printed parts where electrical components require custom mounting or integration, and fabricate the rectangular gate from a transparent material that can be cut and fitted within the project timeline.

The choice of a transparent material for the gate serves both an aesthetic and a practical purpose. Visually, a clear gate gives the station a cleaner, more professional appearance compared to an opaque barrier. Functionally, transparency allows anyone approaching the turnstile to see through the gate to the other side, making it immediately apparent if an obstruction is present that could prevent the gate from opening or completing its rotation. This visibility reduces the risk of mechanical failures going unnoticed during operation.

Regarding actuation, commercial turnstiles rely on solenoid-based locking mechanisms or brushless DC motors driving steel ratchet assemblies, neither of which is practical to replicate within the project constraints. High-torque servo motors are the established alternative in rapid prototyping contexts, providing sufficient stall torque to hold and rotate a lightweight gate reliably. Rather than generating PWM signals directly from the Raspberry Pi 5 GPIO pins, the module uses a PCA9685 servo driver board connected to the Raspberry Pi over I2C, which offloads PWM timing to dedicated hardware. The mechanical connection between the servo output shaft and the gate is handled by a 3D-printed coupling component, which allows the geometry of the joint to be designed precisely in CAD and reprinted quickly if adjustment is needed during development.

2.3 Methodology

The Turnstile Module is organized into two physically distinct parts. The structural part consists of the wooden frame and the transparent rectangular gate, and contains no electrical components. The back side and bottom side of the station are built from wood, providing a rigid and workable base that can be assembled and modified without any dependency on the electronics. The upper part is fully 3D-printed and serves as the control layer, housing the Raspberry Pi 5, the PCA9685 servo driver board, the servo motors, and the associated wiring in a single integrated enclosure designed in CAD. This separation gives both parts the ability to be designed, built, and revised in parallel without one requiring the other to be rebuilt. It is worth noting that additional control sub-units

mounted within the upper layer, such as the RFID reader unit and the visual unit, are described in the IoT Module (MOD-03) documentation.

The data flow through the module is straightforward. The module receives a function call from the IoT Module (MOD-03), either `gate.open()` or `gate.close()`. This call is translated into an I2C command sent from the Raspberry Pi 5 to the PCA9685 servo driver board, which generates the PWM signal at the hardware level and drives the servo motor to the target angle. The servo motor transmits torque through the 3D-printed coupling component to the gate pivot, rotating the rectangular gate to the open or closed position.

Both `gate.open()` and `gate.close()` are provided in two implementations. The primary implementation is written in Python and uses the `adafruit-circuitpython-pca9685` library, which integrates directly with the IoT module since MOD-03 is also implemented in Python and requires no additional language bridging. The secondary implementation is a low-level C++ version that communicates with the PCA9685 directly over I2C without relying on any library, and requires a linkage layer between Python and C++ to be called from MOD-03. Both implementations were completed, so that if the library-based approach fails in the deployment environment, the low-level implementation can be substituted without changes to the physical hardware.

The 3D-printed coupling between the servo output shaft and the gate pivot is a deliberate rapid prototyping decision. The geometry of this joint depends on the final dimensions of the gate, the servo mounting position on the frame, and the rotation angle required to fully clear the passage. Designing this component in CAD and printing it allows the joint geometry to be adjusted and reprinted within hours if the mechanical fit needs correction during assembly, which would not be possible with a machined or off-the-shelf coupling.

Within the overall system, the Turnstile Module is passive for the majority of the inspection cycle. It holds the gate in the locked position from the moment the system enters the IDENTIFYING state until MOD-03 issues a `gate.open()` call on a GRANTED outcome. The gate remains open for a fixed duration to allow the worker to pass, after which `gate.close()` is called automatically by MOD-03 and the system returns to IDLE. The module has no awareness of the inspection logic, the RFID result, or the PPE detection outcome; it only responds to the two control functions it exposes.

2.4 Inter-Module Communication

The Turnstile Module communicates exclusively with the IoT Module (MOD-03). This boundary exists because the Turnstile Module has no role in decision-making. It does not know whether a card was scanned, whether the backend was queried, or whether the AI module detected a missing PPE item. That entire logic chain lives in MOD-03, which acts as the central orchestrator of the inspection cycle. The Turnstile Module is only activated at the moment a final GRANTED or DENIED outcome has already been determined, at which point MOD-03 calls `gate_open()` with a direction parameter indicating whether the worker is entering or exiting, or keeps the gate closed on a DENIED outcome. The `gate_open()` call is blocking: it moves the servo to the open position, waits for a predetermined auto-close delay, and then calls `gate_close()` internally before returning. This design places the timing responsibility entirely inside the Turnstile Module, so MOD-03 does not need to manage a separate timer or issue an explicit close command after every open. The only timing constraint imposed on MOD-03 is that it must not issue a second `gate_open()` call while the first is still blocking, which is naturally enforced by the sequential structure of the inspection state machine.

2.5 Implementation

The module is implemented in two layers that mirror the physical separation described in the Methodology section. The high-level layer is the Python implementation, built around the Turnstile class which wraps the Adafruit ServoKit library. On initialization, the class attempts to connect to the PCA9685 over I2C and immediately drives both servos to the closed position, ensuring the gate is in a known state before the system enters its main loop. The `gate_open()` method accepts a direction argument and sets both servo channels to the appropriate angles for that direction before blocking

for the sweep duration, holding the gate open for the auto-close delay, and then calling `gate_close()` internally. If the PCA9685 is not reachable at initialization, the class sets an internal flag and skips all subsequent hardware calls without raising an exception, which allows the rest of the system to continue running in a degraded state during development without physical hardware present.

The low-level layer is the C++ implementation, where `gate_open()` and `gate_close()` are built on top of a dedicated servo driver that handles direct I2C communication with the PCA9685. When `gate_open()` is called with a direction, it selects the appropriate servo channel for that direction, commands the servo driver to move to the target angle, blocks for the sweep duration, holds for the auto-close delay, and then calls `gate_close()` internally. When `gate_close()` is called, it returns both servo channels to their resting positions and then disables the PWM output on each channel to eliminate the holding current drawn by the MG995 at stall. The gate control layer maintains an observable state across these transitions, moving through CLOSED, MOVING, and OPEN states, and exposes an optional callback that MOD-03 can register to be notified on each state change without polling.

The two implementations share the same behavioral contract defined in the module headers, which allowed them to be developed and verified independently before the physical enclosure was printed. The Python implementation is the primary path consumed by MOD-03 since the IoT module is also written in Python and no language bridging is required. The C++ implementation is available as a fallback if the Adafruit library fails in the deployment environment, in which case a thin binding layer between Python and C++ allows MOD-03 to call it through the same interface without changes to the IoT module's logic.

2.6 Testing Strategy

The Turnstile Module is tested in two stages. The first stage is unit testing of the Python implementation without physical hardware, using the internal flag set when the PCA9685 is not reachable to verify that the module initializes and handles calls to `gate_open()` and `gate_close()` without crashing. The second stage is hardware-in-the-loop testing on the physical assembly, where each test case is verified by visual observation of the gate arm movement. The acceptance criteria for this module are that the gate opens fully to the correct position for each direction, holds for the auto-close delay, and returns to the closed position without manual intervention.

Test ID	Scenario	Input	Expected Output	Pass Criteria
T-0 1	Gate opens for entry	<code>gate_open(DIR_ENTRY)</code>	Entry servo moves to open position, exit servo unchanged	Gate arm clears the passage fully
T-0 2	Gate opens for exit	<code>gate_open(DIR_EXIT)</code>	Exit servo moves to open position, entry servo unchanged	Gate arm clears the passage fully
T-0 3	Gate closes after auto-close delay	<code>gate_open()</code> called, no manual close	Gate closes automatically after the predetermined delay	Both arms return to resting position
T-0 4	Explicit gate close	<code>gate_close()</code>	Both servos return to closed position	Gate arm blocks the passage
T-0 5	Invalid direction argument	<code>gate_open("INVALID")</code>	Call is ignored, gate remains	No servo movement, no exception raised

			closed, warning is logged	
T-0 6	Hardware not available	PCA9685 disconnected at init	Module initializes with unavailable flag, all calls skipped silently	No exception raised, system continues
T-0 7	Gate already closed	gate_close() called when gate is already closed	No servo movement, returns success	Gate remains closed, no mechanical jitter

2.7 Known Limitations and Open Issues

The most significant limitation of the current implementation is that `gate_open()` is a blocking call. While the gate is open and waiting for the auto-close delay to expire, the Raspberry Pi thread that called `gate_open()` is held, meaning MOD-03 cannot process a new RFID scan or respond to any other system event during that window. This is acceptable for the current single-worker assumption of the system but would become a practical problem if two workers approached the turnstile in quick succession. A non-blocking implementation using a dedicated timer thread is noted as a future consideration in the module headers but has not been implemented, which corresponds to a scheduling risk in the integration phase.

The physical design also carries an unverified assumption at the time of writing. The torque rating of the MG995 servo has been selected based on the estimated weight of the transparent gate panel, but the actual mechanical load has not been measured against the servo's rated torque under the final fabricated dimensions. If the gate panel is heavier than estimated, or if the 3D-printed coupling introduces additional friction at the pivot, the servo may fail to complete its sweep reliably, which corresponds to risk R-06 in the project risk register. This will be verified during the hardware-in-the-loop testing phase in Week 6, and the coupling geometry or servo selection will be revised if needed.

3. MOD-03 IoT Module

3.1 Module Overview

The IoT module serves as the central hardware and software coordinator running on the Raspberry Pi 5 platform. It orchestrates the access control state machine by sequentially interacting with the SPI-connected RC522 RFID reader, executing real-time PPE detection via the Hailo-8L NPU (MOD-01), fetching worker identity and validation rules from the remote Node.js backend (MOD-04), and communicating physical access decisions directly to the native Raspberry Pi GPIO servomotor gate hardware (MOD-02). Furthermore, it broadcasts asynchronous, real-time visual system state updates to the React turnstile display application (MOD-05) via WebSockets. This module acts as the physical edge node where all sensor inputs, AI inferences, and mechanical outputs converge into a unified worker inspection cycle. By centralizing the control logic at the edge, the system maintains strict latency bounds while reducing reliance on external network availability during the critical inspection phase. It ensures that any mechanical or visual state change is directly driven by a verified set of hardware-accelerated detection results and centralized permissions, establishing a robust access control flow.

3.2 Background and Related Work

Access control systems typically rely on polling architectures or single-purpose microcontrollers that struggle with the high-bandwidth requirements of real-time computer vision. The team investigated using a lightweight microcontroller (such as an ESP32) to handle peripheral I/O while offloading AI processing to the cloud. However, this approach introduces unacceptable inference latency and couples physical access reliability to continuous external network uptime, which violates the strict 10-second inspection cycle constraint.

To achieve low-latency inference while maintaining robust peripheral control, edge computing platforms were evaluated. Integrating the AI object detection model directly into the same local physical node as the mechanical actuators eliminates network round-trip overhead during the critical inspection phase. Consequently, the Raspberry Pi 5 combined with the Hailo-8L hardware accelerator was selected because it provides sufficient computational headroom for edge AI inference while exposing native GPIO, SPI, and I2C buses to interface directly with the turnstile sensors and motors.

A further hardware architectural decision concerns the physical placement and power independence of the RFID reader unit. Rather than routing the RC522 reader's SPI signals directly to the Raspberry Pi 5 over extended cabling — which would introduce signal integrity risks across the turnstile structure — the team opted to pair the RC522 module with a dedicated ESP32 microcontroller and a compact lithium-ion battery pack. This configuration makes the RFID reader unit entirely self-contained and cable-independent, allowing it to be mounted directly on the turnstile body at an ergonomically accessible position without requiring a physical tether to the central processing node. The ESP32 reads the UID from the RC522 over SPI and forwards it wirelessly to the Raspberry Pi 5 orchestrator. To house this embedded assembly in a mechanically robust and weathering-resistant enclosure suitable for an industrial entry point, the outer casing was fabricated using fused deposition modeling (FDM) 3D printing. This manufacturing approach was selected for its low per-unit cost, rapid design iteration capability, and the ability to produce custom geometries that conform precisely to the chosen mounting surface on the turnstile frame.

3.3 Methodology

The IoT module is internally structured around a synchronous, single-threaded state machine orchestrator alongside an asynchronous communication bridge. The `IoTOrchestrator` class acts as the primary controller, advancing through strict sequence phases: IDLE, IDENTIFYING, INSPECTING, and DENIED or GRANTED. Hardware and network dependencies, such as the RFID reader, gate controller, backend client, and display notifier, are injected into the orchestrator during initialization. This dependency injection pattern is chosen because it isolates the core state machine

from hardware volatility, permitting components to be mocked and validated rigorously in software isolation.

Data enters the module physically as SPI signals from the RC522 RFID reader, which the hardware layer decodes into a UID string. This UID is transmitted via HTTP REST to the remote backend, returning a structured JSON payload defining the worker's required PPE. The module then captures an image frame from the locally connected Raspberry Pi Camera v3 via OpenCV and passes it to the AI vision module (MOD-01) for inference. The inference output, an array of detected PPE strings, is logically compared against the backend requirements. The resulting boolean decision terminates the flow by dispatching GPIO PWM signals to the gate controller and emitting the final status packet to connected displays.

A key design decision involves the decoupling of the turnstile display updates (MOD-05) from the synchronous access logic. The team considered having the orchestrator directly push HTTP updates to the display, but this was rejected because network delays would block the primary thread, potentially stalling the physical gate mechanism. Instead, an independent daemon thread running an `asyncio` event loop manages a `WebSocketDisplayNotifier`. The orchestrator executes fire-and-forget state notifications to this parallel thread, ensuring that network slow-downs or disconnected tablets never impede or disrupt the critical path of the inspection cycle.

The module remains dormant in the IDLE state until motion or an RFID badge is presented. Once a badge is detected, the module locks the process, traverses the identity and inference stages, commands the mechanical barrier, pushes audit logs to the backend database via HTTP POST, and finally re-enters the IDLE state, ready for the next worker.

3.4 Inter-Module Communication

MOD-01 (AI and Vision): The IoT module communicates with the AI module to perform localized hardware-accelerated PPE detection. The boundary exists because the IoT orchestrator is responsible for capturing the physical camera frame at the correct moment in the access flow, but it delegates the computationally intensive YOLOv8n execution to the specialized AI component. Without this communication, the system could identify workers but would possess no capability to verify their compliance with safety requirements.

MOD-02 (Turnstile): The IoT module interfaces with the Turnstile module to translate logical access decisions into tangible physical actions. This boundary isolates high-level software validation from low-level electromechanical PWM signal generation. If this boundary failed, an approved worker would remain physically barred from entry, negating the purpose of the inspection station.

MOD-04 (Backend and Database): The IoT module queries the remote backend to resolve RFID UIDs into specific worker roles and explicit PPE enforcement rules. This network boundary is mandated because the central database serves as the single source of truth for dynamically changing personnel profiles across multiple facility entrances, preventing the Pi from relying on stale, hardcoded rules. Furthermore, the orchestrator uploads inspection outcomes back across this boundary to provide persistent, centralized compliance logging for administrators.

MOD-05 (UI and UX Display): The IoT module pushes real-time state transition events (such as "Scanning", "Pass", or missing PPE lists) to the React display application via WebSockets. This boundary is necessary to provide immediate, localized visual feedback to the worker standing at the gate, instructing them on how to correct their attire if entry is denied.

3.5 Implementation

The gate control mechanism avoids utilizing a separate C/C++ micro-service and instead directly drives the servomotors via Python generating native GPIO PWM signals directly from the Raspberry Pi. This optimization decreases inter-process communication latency on the Pi and eliminates reliance on an external driver board. The implementation enforces strict mapping of signal pulse

widths (0.5 ms for 0 degrees, 2.5 ms for 180 degrees at 50 Hz) corresponding directly to the locked and open states of the gate arms.

To bridge the gap between differing schema layouts, the display implementation implements a data translation layer. When the backend returns a worker profile containing integer PPE identifiers without visual metadata, the IoT module cross-references these internal identifiers and injects fallback display strings before broadcasting to the WebSocket. This ensures that the tablet UI never receives malformed rendering instructions, preventing UI crashes even if the backend returns unrecognized item keys. Furthermore, the WebSocket server is configured to immediately broadcast the most recently cached state payload to any newly connected client. This specific recovery implementation handles edge cases where the tablet display loses Wi-Fi connectivity and reconnects mid-inspection, allowing the screen to instantly catch up to the hardware gate's reality without prompting the worker to rescan their card.

3.6 Testing Strategy

Test ID	Scenario	Input	Expected Output	Pass Criteria
IOT-01	Valid RFID and full PPE	Registered UID, correct PPE identified by AI	Transitions logically from IDLE to IDENTIFYING to INSPECTING. Gate opens.	gate_open() invoked, PASS logged with backend, re-enters IDLE.
IOT-02	Valid RFID, missing PPE	Registered UID, missing safety vest in vision frame	Gate stays closed, show_denied(["safety_vest"]) invoked via WebSocket.	gate_open() is never called, FAIL logged with required vs missed PPE arrays.
IOT-03	Unknown RFID Card	Unregistered UID	Halts flow before inspection, transitions to UNKNOWN_CARD state.	Inference skipped, backend audit logs UNKNOWN_CARD, re-enters IDLE after timeout.
IOT-04	WebSocket disconnect	Client drops during INSPECTING state	Cycle continues uninterrupted, makes accurate access decision.	Output relies entirely on AI result, physical gate is commanded correctly.

3.7 Known Limitations and Open Issues

The primary hardware limitation of the current implementation involves the physical tuning of the gate controller. The structural tolerances of the 3D-printed chassis and the required torque for the dual servo arms under real-world physical load remain unverified, meaning the strict rotational angles (e.g., exactly 90 degrees for open) currently utilized by the GPIO PWM logic are theoretical placeholders. Additionally, the system currently assumes constant indoor lighting; the AI vision wrapper does not possess mechanisms for dynamic exposure adjustment on the camera, risking severe accuracy degradation under variable sunlight conditions (Risk R-03).

4. MOD-04 Backend & Database Module

4.1 Module Overview

The Backend and Database module serves as the central data persistence and business logic layer for the AI-Powered Smart PPE Inspection Station. Running on a Node.js 20 runtime with an Express.js 5 framework, it exposes a RESTful API to manage workers, roles, and safety compliance logs. It persists all operational data in a PostgreSQL relational database. The backend directly interacts with the IoT module, which pushes real-time inspection results, and the UI and UX module, which fetches configuration data and compliance statistics. By centralizing validation and storage, it acts as the authoritative source of truth in the system data flow.

4.2 Background and Related Work

The team investigated multiple database paradigms, explicitly comparing NoSQL options with traditional relational databases like PostgreSQL. A relational database is selected because the problem domain requires strict relational integrity between registered workers, their assigned roles, and the specific personal protective equipment mandated for each role. PostgreSQL provides ACID compliance, which guarantees that entry transactions and their associated PPE detection details are saved atomically, preventing corrupt or partial compliance records.

The team also evaluated full-stack Node.js frameworks such as NestJS against minimalistic micro-frameworks like Express.js. Express.js is chosen because it carries a significantly lighter footprint and avoids the steep learning curve and boilerplate overhead of dependency injection containers. This allows the backend to remain tightly focused on serving the IoT module's REST API requests quickly without unnecessary software layers.

Additionally, raw SQL queries were compared against Object-Relational Mapping libraries. The Prisma ORM is adopted because it automatically generates type-safe database clients from a declarative schema model. This eliminates manual SQL syntax errors and seamlessly maps relational query results into nested JSON structures, reducing the controller logic required when communicating with the React-based UI and UX module.

4.3 Methodology

The backend module is organized into a modular layered architecture consisting of routes, controllers, and Prisma data models. External HTTP requests are first filtered by Express middlewares that apply security headers via Helmet and enable Cross-Origin Resource Sharing. Valid requests are forwarded to endpoint-specific controller functions. The controllers extract payloads, apply business logic, and invoke the generated Prisma client to perform the necessary persistence actions. This structure is chosen because separating the HTTP routing from the business logic ensures that each component can be independently tested with mocked database inputs.

Data flows into the module exclusively via JSON payloads over HTTP REST. When a payload arrives at an endpoint, the controller validates the presence and type of required fields. The controller then translates the request into Prisma ORM commands, which convert the operation into parameterized raw SQL queries executed by the PostgreSQL engine. Once the database operation completes, the data flows back out to the client formatted as a structured JSON response containing the success status and the manipulated records.

A key design decision involves how inspection results are persisted. Instead of issuing independent database insert commands for a scan event and its individual equipment detections, the module wraps them in a single Prisma transaction. A transaction is used because physical hardware failures or network interruptions could occur precisely while a log is being recorded. If a failure happens, the transaction rolls back entirely, which guarantees that the database never contains floating detection records without a parent entry log.

The module operates globally and continuously behind the scenes to bridge the physical inspection station with human administrators. It remains persistently active, waiting for the active state machine of the IoT module to complete a cycle and transmit the outcome. Before it runs, states like worker roles exist in the database ready to be queried; after an API call completes, the system holds highly structured, persistently stored records of the gate's decision history.

4.4 Inter-Module Communication

The backend communicates with the IoT module to enforce physical security and securely persist inspection outcomes. The IoT module depends on the backend to evaluate the scanned RFID card UID and return the worker's required PPE list. Without this communication, the IoT module would lack the specific rules needed to operate its inference engine correctly. This responsibility boundary exists because centralizing the business rules on the backend prevents the hardware station from needing frequent rule synchronization or local storage provisioning.

The backend also communicates with the UI and UX module to surface compliance insights to human administrators. The UI application depends on the backend to retrieve both granular tracking data and aggregated statistics, such as compliance rates and frequently missed equipment. This communication is strictly required so that safety administrators can actively configure the physical rules of the system from a remote PC interface, effectively decoupling the administrative control loop from the physical turnstile environment.

4.5 Implementation

A non-obvious implementation challenge involves the graceful handling of unregistered personnel arriving at the inspection station. When the IoT module scans an unrecognized RFID card, it generates an entry log with an outcome of UNKNOWN_CARD. The backend implementation accommodates this by relaxing the worker relation constraint on the entry log table, allowing the worker ID to be completely null. This design satisfies the database foreign key requirements while ensuring that the station captures the exact time and camera snapshot URL of the illicit entry attempt for auditing purposes.

Another critical implementation detail is the dynamic calculation of missing personal protective equipment within the entry log controller. When an inspection is graded as a failure, the controller filters the saved detection details to identify items that were mandated by the worker's role but not detected by the AI vision models. It maps these missing items out dynamically and embeds them in the raw JSON response payload. This computationally trivial step prevents the calling client from needing to re-fetch the role requirements and perform a diff logic calculation on its side.

Finally, the backend implements database model mapping to synthesize strict database conventions with modern JavaScript paradigms. It utilizes the Prisma schema mapping directive to define tables and columns in standard snake case structure entirely within the PostgreSQL database representation. However, the generated Javascript code exposes these identical entities strictly using camel case identifiers. This implementation decision allows Node.js developers to maintain standard casing conventions in the application layer without compromising conventional database administrator naming schemes.

4.6 Testing Strategy

The module is tested with Jest using two complementary layers: HTTP integration tests with Supertest for all exposed backend routes, and direct unit tests for shared middleware behaviors. Tests run during local development and before integration updates, with Prisma fully mocked so controller and middleware logic can be validated deterministically without provisioning a live PostgreSQL instance. The acceptance criteria require that each implemented route and middleware branch returns the expected status code or control-flow behavior, that validation failures produce the documented error structure, and that Prisma error paths are converted into stable server-error responses.

Test ID	Scenario	Input	Expected Output	Pass Criteria
TST-001	Health endpoint returns service heartbeat	GET /api/health	HTTP 200 with status, timestamp, and numeric uptime	Response status is 200 and all three fields exist with correct types.
TST-002	Root endpoint returns runtime banner	GET /	HTTP 200 with plain text API-running message	Response body exactly matches configured startup message.
TST-003	Worker list retrieval	GET /api/workers	HTTP 200 with success: true, worker array, and total	One mocked worker is mapped to snake_case response fields and count is returned.
TST-004	Worker list filter by active flag	GET /api/workers?is_active=false	HTTP 200 with filtered query path exercised	Prisma findMany receives where.isActive = false.
TST-005	Worker list filter by role id	GET /api/workers?role_id=2	HTTP 200 with role filter path exercised	Prisma findMany receives where.roleId = 2.
TST-006	Worker creation success	POST /api/workers with valid full_name, rfid_card_uid, role_id	HTTP 201 with created worker payload	Response includes success: true and created full_name.
TST-007	Worker creation rejects missing fields	POST /api/workers missing rfid_card_uid and role_id	HTTP 422 validation error	Error envelope returns success: false and error.code = 422.
TST-008	Worker creation rejects duplicate RFID	POST /api/workers with existing rfid_card_uid	HTTP 409 conflict	Error message indicates RFID duplication.
TST-009	Worker creation rejects unknown role	POST /api/workers with non-existent role_id	HTTP 404 role-not-found error	Error message states role is not found.
TST-010	Worker detail retrieval by id	GET /api/workers/1	HTTP 200 with worker detail payload	Response contains success: true and data.id = 1.

Test ID	Scenario	Input	Expected Output	Pass Criteria
TST-011	Worker detail returns 404 for unknown id	GET /api/workers/999	HTTP 404 worker-not-found error	Error message indicates worker is not found.
TST-012	Worker update success	PUT /api/workers/1 with full_name change	HTTP 200 with updated worker payload	Response includes updated full_name.
TST-013	Worker update returns 404 for unknown id	PUT /api/workers/999	HTTP 404 worker-not-found error	Status code is 404 when target worker does not exist.
TST-014	Worker update rejects duplicate RFID	PUT /api/workers/1 with already-used rfid_card_uid	HTTP 409 conflict	Error message indicates RFID duplication on update.
TST-015	Worker soft delete success	DELETE /api/workers/1	HTTP 200 with deactivation message and is_active: false	Response confirms deactivation and worker remains in payload as inactive.
TST-016	Worker soft delete returns 404 for unknown id	DELETE /api/workers/999	HTTP 404 worker-not-found error	Status code is 404 when target worker does not exist.
TST-017	RFID card lookup returns worker and required PPE	GET /api/workers/card/ABC123 with registered card	HTTP 200 with worker and required_ppe[]	Response includes worker identity and mapped PPE item_key entries.
TST-018	RFID card lookup returns 404 for unknown card	GET /api/workers/card/UNKNOWN	HTTP 404 card-not-registered error	Error message indicates card is not registered.
TST-019	Worker list handles Prisma failure	GET /api/workers with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-020	Worker creation handles Prisma failure	POST /api/workers with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-021	Worker detail handles Prisma failure	GET /api/workers/:id with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.

Test ID	Scenario	Input	Expected Output	Pass Criteria
TST-022	Worker update handles Prisma failure	PUT /api/workers/:id with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-023	Worker delete handles Prisma failure	DELETE /api/workers/:id with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-024	Worker card lookup handles Prisma failure	GET /api/workers/card/:uid with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-025	Role list retrieval	GET /api/roles	HTTP 200 with role array	Response maps role fields to snake_case output.
TST-026	Role creation success	POST /api/roles with valid role_name	HTTP 201 with created role payload	Response includes success: true and created role_name.
TST-027	Role creation rejects missing role_name	POST /api/roles without role_name	HTTP 422 validation error	Error message indicates role_name is required.
TST-028	Role creation rejects duplicate role_name	POST /api/roles with existing role_name	HTTP 409 conflict	Error message indicates role already exists.
TST-029	Role PPE requirement retrieval	GET /api/roles/1/ppe	HTTP 200 with role identity and ppe_items[]	Response includes role metadata and PPE list mapping.
TST-030	Role PPE retrieval returns 404 for unknown role	GET /api/roles/999/ppe	HTTP 404 role-not-found error	Error message indicates role is not found.
TST-031	Role PPE requirement update success	PUT /api/roles/1/ppe with ppe_item_ids: [1]	HTTP 200 with updated PPE list	Response includes one updated PPE item entry.
TST-032	Role PPE update rejects invalid ppe_item_ids type	PUT /api/roles/1/ppe with ppe_item_ids: "not-an-array"	HTTP 422 validation error	Error message indicates ppe_item_ids must be an array.

Test ID	Scenario	Input	Expected Output	Pass Criteria
TST-033	Role PPE update rejects missing ppe_item_ids	PUT /api/roles/1/ppe with empty body	HTTP 422 validation error	Status 422 returned when ppe_item_ids is absent.
TST-034	Role PPE update returns 404 for unknown role	PUT /api/roles/999/ppe	HTTP 404 role-not-found error	Error message indicates role is not found.
TST-035	Role list handles Prisma failure	GET /api/roles with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-036	Role creation handles Prisma failure	POST /api/roles with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-037	Role PPE retrieval handles Prisma failure	GET /api/roles/:id/ppe with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-038	Role PPE update handles Prisma failure	PUT /api/roles/:id/ppe with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-039	PPE catalog retrieval	GET /api/ppe-items	HTTP 200 with PPE item array	Response maps PPE fields to snake_case output.
TST-040	PPE detail retrieval by id	GET /api/ppe-items/1	HTTP 200 with PPE item detail	Response includes expected item_key.
TST-041	PPE detail returns 404 for unknown id	GET /api/ppe-items/999	HTTP 404 PPE-not-found error	Error message indicates PPE item is not found.
TST-042	PPE item creation success	POST /api/ppe-items with valid item_key, display_name, icon_name	HTTP 201 with created PPE payload	Response includes success: true and created item_key.
TST-043	PPE item creation rejects partial missing fields	POST /api/ppe-items missing display_name	HTTP 422 validation error	Error message indicates display_name is required.
TST-044	PPE item creation rejects fully missing fields	POST /api/ppe-items with empty body	HTTP 422 validation error	Error message indicates required fields such as item_key.

Test ID	Scenario	Input	Expected Output	Pass Criteria
TST-045	PPE item creation rejects duplicate item_key	POST /api/ppe-items with existing item_key	HTTP 409 conflict	Error message indicates item_key already exists.
TST-046	PPE item update success	PUT /api/ppe-items/1 with changed display_name	HTTP 200 success response	Response returns success: true on update path.
TST-047	PPE item update returns 404 for unknown id	PUT /api/ppe-items/999	HTTP 404 PPE-not-found error	Status code is 404 when target PPE item does not exist.
TST-048	PPE item update rejects duplicate item_key	PUT /api/ppe-items/1 with already-used item_key	HTTP 409 conflict	Error message indicates duplicate item_key.
TST-049	PPE item delete success	DELETE /api/ppe-items/1 not referenced by roles	HTTP 200 with delete confirmation	Response includes deletion message and deleted item id.
TST-050	PPE item delete returns 404 for unknown id	DELETE /api/ppe-items/999	HTTP 404 PPE-not-found error	Status code is 404 when target PPE item does not exist.
TST-051	PPE item delete blocked when referenced by roles	DELETE /api/ppe-items/1 with role usage count > 0	HTTP 409 conflict with usage count message	Error message explains deletion is blocked by role assignments.
TST-052	PPE catalog retrieval handles Prisma failure	GET /api/ppe-items with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-053	PPE detail retrieval handles Prisma failure	GET /api/ppe-items/:id with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-054	PPE item creation handles Prisma failure	POST /api/ppe-items with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-055	PPE item update handles Prisma failure	PUT /api/ppe-items/:id with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.

Test ID	Scenario	Input	Expected Output	Pass Criteria
TST-056	PPE item delete handles Prisma failure	DELETE /api/ppe-items/:id with Prisma throw	HTTP 500 server error envelope	Response returns success: false with status 500.
TST-057	Entry log creation success for PASS result	POST /api/entry-logs with valid PASS payload and detections	HTTP 201 with entry_log_id, result, missing_ppe: []	Response confirms created log and empty missing PPE list for pass case.
TST-058	Entry log FAIL response includes missing PPE	POST /api/entry-logs with result: FAIL and undetected required PPE	HTTP 201 with missing_ppe[] including missed item	Response includes expected missed PPE item_key values.
TST-059	Entry log creation rejects missing RFID UID	POST /api/entry-logs without rfid_uid_scanned	HTTP 422 validation error	Error message indicates rfid_uid_scanned is required.
TST-060	Entry log creation rejects invalid result enum	POST /api/entry-logs with result: INVALID	HTTP 422 validation error	Error message states allowed result values.
TST-061	Entry log creation rejects non-array detections	POST /api/entry-logs with detections: "not-array"	HTTP 422 validation error	Error message indicates detections must be an array.
TST-062	Entry log creation rejects missing detections	POST /api/entry-logs without detections	HTTP 422 validation error	Error message indicates detections field is required.
TST-063	Entry log list retrieval with default pagination	GET /api/entry-logs	HTTP 200 with log array, total, default limit=50, offset=0	Response includes mapped worker name and default pagination metadata.
TST-064	Entry log list filter by worker and result	GET /api/entry-logs?worker_id=1&result=FAIL	HTTP 200 with filtered query path exercised	Prisma findMany receives where.workerId = 1 and where.result = FAIL.
TST-065	Entry log list applies custom limit and offset	GET /api/entry-logs?limit=10&offset=5	HTTP 200 with paginated metadata	Prisma query uses take: 10, skip: 5, and response

Test ID	Scenario	Input	Expected Output	Pass Criteria
				echoes same values.
TST-066	Entry log list applies date range filter	GET /api/entry-logs?start_date=2026-04-01&end_date=2026-04-10	HTTP 200 with date-filter path exercised	Prisma where.scannedAt contains valid gte and lt date bounds.
TST-067	Entry log stats aggregates compliance and misses	GET /api/entry-logs/stats	HTTP 200 with counts, compliance rate, and most_missed_ppe[]	Response returns total/passed/failed/unknown counts and computed compliance (~84.2%).
TST-068	Entry log stats handles zero denominator	GET /api/entry-logs/stats with no pass/fail records	HTTP 200 with compliance_rate: 0	Compliance rate is exactly 0 and missed PPE list is empty.
TST-069	Entry log stats accepts period parameters	GET /api/entry-logs/stats?start_date=2026-04-01&end_date=2026-04-10	HTTP 200 with echoed period object	Response period matches provided start and end dates.
TST-070	requireFields passes when all fields exist	Middleware invocation with body: {name, email} and required ['name','email']	Calls next() and does not write response	next is called exactly on valid input path.
TST-071	requireFields rejects missing field	Middleware invocation with missing email	Writes HTTP 422 error payload	Response contains error.code = 422 and message email is required.
TST-072	requireFields rejects null field value	Middleware invocation with name: null	Writes HTTP 422 error payload	Response contains message name is required.
TST-073	requireFields rejects empty string field value	Middleware invocation with name: "	Writes HTTP 422 error payload	Status 422 is returned for empty required string values.
TST-074	requireFields passes when no required keys configured	Middleware invocation with required []	Calls next()	next is called without validation failures.

Test ID	Scenario	Input	Expected Output	Pass Criteria
TST-075	errorHandler maps unhandled error to 500	Middleware invocation with <code>Error('Something broke')</code>	Writes HTTP 500 error payload	Response contains <code>error.code = 500</code> and original message.
TST-076	errorHandler honors explicit <code>err.statusCode</code>	Middleware invocation with error carrying <code>statusCode = 404</code>	Writes HTTP 404 error payload	Response code follows <code>err.statusCode</code> precedence.
TST-077	errorHandler honors pre-set non-200 response code	Middleware invocation with <code>res.statusCode = 400</code> and thrown error	Writes HTTP 400 error payload	Response status remains 400 instead of falling back to 500.
TST-078	errorHandler applies default internal error message	Middleware invocation with empty error object	Writes error payload with default message	Response message is <code>Internal server error</code> when no message exists.

4.7 Known Limitations and Open Issues

A known limitation of the backend implementation is the complete absence of authentication or authorization middleware on the exposed routes. The system fundamentally trusts all incoming requests, assuming they originate within a secure internal network environment. This limitation means an attacker with local network access could easily forge entry logs or tamper with the worker databases. Furthermore, the endpoints lack rate limiting, which exposes the Prisma application layer to potential stress if the hardware module malfunctions and rapidly spams API requests. Lastly, project risk R-07 details concerns regarding the physical Wi-Fi reachability between the backend environment and the Raspberry Pi deployment location.

5. MOD-05 UI & UX Module

5.1 Module Overview

The UI and UX module provides the two human-facing interfaces of the PPE Inspection Station. The first is the Admin Panel, a React 18 web application that runs on a standard PC browser and is used by safety managers for worker registration, role configuration, log review, and compliance reporting. The second is the Turnstile Display, a React 19 application that runs on a dedicated tablet at the physical gate and presents real-time inspection feedback to the worker standing in front of the turnstile. The Admin Panel communicates exclusively with MOD-04 Backend over HTTP REST, while the Turnstile Display receives inspection state updates from MOD-03 IoT over a WebSocket connection. Because the two applications run on different hardware, serve different users, depend on different network protocols, and require different third-party libraries, they are implemented as two independent Vite projects under the same repository. The remainder of this section documents each sub-module separately.

5.A Admin Panel

5.A.1 Background and Related Work

Industrial access control dashboards have traditionally been built as desktop applications or server-rendered web pages tightly coupled to a specific database vendor. The limitation of that approach is that every change in the operational workflow, such as adding a new PPE category, requires a full server redeployment. Modern component-based frontend frameworks decouple the user interface from the backend entirely, which is why the Admin Panel uses React 18 consuming a REST API rather than a server-rendered template. This choice was made early so that the frontend and the backend could be developed in parallel against a shared API contract.

Two alternatives to React were considered: Vue.js and a plain HTML and vanilla JavaScript stack. Vue.js was rejected because the team had prior experience only with React and the project timeline did not allow for learning a new framework. Plain HTML with vanilla JavaScript was rejected because the Admin Panel requires five distinct routed pages with shared state, and managing cross-page state without a framework would require reimplementing patterns that React already provides. Vite was selected over Create React App because Create React App has been deprecated by its maintainers. Tailwind CSS was chosen over CSS modules because the Admin Panel contains many small styled elements such as stat cards, badges, table rows, and form inputs, and Tailwind keeps all style information in the JSX file next to the markup, making each component self-contained. Vitest was chosen over Jest because the project already uses Vite, and Vitest reuses the Vite configuration directly without requiring a separate Babel setup.

5.A.2 Methodology

The Admin Panel is organized into five top-level pages: Dashboard, Workers, Roles and PPE, Entry Logs, and Analytics. Each page is implemented as a single React component file under `src/pages/`. A shared Layout component provides the sidebar navigation and renders the current route using React Router v6. All data fetching is centralized in a single service module at `src/services/api.js`, which exposes one async function per REST endpoint. Pages never call `fetch` directly; they only call the service functions. This separation exists because MOD-04 is under active development and the service module currently returns mock data from in-memory arrays. When MOD-04 is complete, only the service module needs to change and no page-level code needs to be modified.

A page component mounts, triggers a `useEffect` hook that calls one or more service functions, and stores the resulting JSON in local state using `useState`. The page then renders the state as a table, chart, or form. When the user performs an action such as submitting a registration form or clicking a delete button, the page calls the relevant service function and either updates local state directly or

refetches to get the latest data. All service functions return data in the shape defined by `admin_side_interface.d.ts`, which is the shared interface contract agreed with MOD-04.

The decision to use a mock-first approach rather than waiting for the backend was made to allow the two efforts to proceed in parallel. The mock data is written to match the exact JSON shapes specified in the API contract, including the paginated response envelope and the object-array form of required PPE items. Aligning the Admin Panel mocks with the object-array form early surfaced the cross-module incompatibility documented as issue C-07 in the compatibility notes. The Admin Panel activates whenever an administrator opens its URL in a browser on the local network and has no runtime dependency on the inspection loop. Its only state-changing effect on the rest of the system is through backend writes, which MOD-03 then reads during the next card lookup.

5.A.3 Inter-Module Communication

The Admin Panel communicates directly only with MOD-04 Backend. Every page-level action, including worker registration, role PPE reassignment, log filtering, and statistics retrieval, is expressed as a call to one of the REST endpoints listed in the module interface document. This communication boundary exists because MOD-04 owns the single source of truth for all worker and role data, and placing a local database inside the Admin Panel would create two authoritative copies of the same data that would drift out of sync. The direction of dependency is one-way: the Admin Panel depends on MOD-04, but MOD-04 does not depend on the Admin Panel, so the backend can continue serving MOD-03 inspection requests even if the Admin Panel is offline.

The Admin Panel has no direct connection to MOD-03, MOD-01, or MOD-02. This boundary is intentional because the Admin Panel is a management interface, not a real-time control interface, and a direct connection to MOD-03 would place inspection-critical traffic on the same network path as administrative traffic. Any inspection data the Admin Panel needs to display, such as entry log history or compliance statistics, reaches it indirectly: MOD-03 writes entry logs to MOD-04 after each inspection and the Admin Panel reads those logs from MOD-04 on demand. The delay between a gate event and its appearance in the Admin Panel is equal to however long MOD-03 takes to POST the log, typically under one second, which is acceptable because administrators do not need real-time freshness on historical records.

5.A.4 Implementation

The most non-obvious implementation detail in the Admin Panel is the handling of the paginated entry logs response. The backend returns data in an envelope where the `total` field holds the count before pagination is applied. The Logs page uses `total` to compute the number of pages and renders a pagination control, while the data array contains only the rows for the current page. The component tracks the current offset in component state and re-fetches when the user changes pages, rather than fetching all logs at once, because the entry log table is expected to grow continuously and loading all rows would become slower over time.

The Roles and PPE page implements the full-replace semantics of the role PPE update endpoint, which overwrites the existing PPE requirement list rather than merging with it. The implementation sends the complete list of currently checked PPE item IDs on every save, including items that were already present before the user edited the form. The only edge case handled specifically is the empty list: submitting an empty array removes all PPE requirements from a role, which is useful for visitor roles that require no protective equipment.

The Workers page handles the RFID card registration flow without a physical reader attached to the administrator's PC. For the alpha release, a Scan Card button in the registration modal generates a random eight-character hexadecimal UID and inserts it into the read-only UID field. This substitution exists because wiring a second RFID reader to the administrator's PC is out of scope for the alpha phase. When the admin-side reader is added later, only the scan button's handler function changes; the rest of the form including validation and the POST request remains unchanged. Worker search and filtering is performed in JavaScript on the frontend after the full worker list is fetched, rather than

through a backend query parameter, because the expected worker count for the demonstration is under 50.

5.A.5 Testing Strategy

The Admin Panel uses Vitest as its test runner with React Testing Library for component rendering tests and jest-dom matchers for DOM assertions. All tests run against the mock API service rather than a live backend, which makes them deterministic and independent of MOD-04's development schedule. Tests are run locally before every commit, and the acceptance criterion for alpha completion is that all tests pass and every page renders without runtime errors when navigated from the sidebar. The test suite contains 41 tests split between the API service layer and the page components.

Test ID	Scenario	Input	Expected Output	Pass Criteria
T-0 5-0 1	API returns worker list	Call <code>api.getWorkers()</code>	Non-empty array of Worker objects	Array length > 0, each item has id, fullName, rfidCardUid, roleId
T-0 5-0 2	API returns worker by ID	Call <code>api.getWorker(1)</code>	Single Worker object with id=1	Returned object has id === 1, fullName is a string
T-0 5-0 3	Non-existent worker	Call <code>api.getWorker(9999)</code>	null returned	Function returns null, does not throw
T-0 5-0 4	API creates worker	Call <code>api.createWorker({fullName, rfidCardUid, roleId})</code>	New Worker object with isActive=true	Returned object matches input and includes a generated id
T-0 5-0 5	API soft-deletes worker	Call <code>api.deleteWorker(1)</code>	Success flag, isActive becomes false	Subsequent <code>getWorkers()</code> shows id=1 with isActive === false
T-0 5-0 6	Filter logs by result	Call <code>api.getEntryLogs({result: 'FAIL'})</code>	Array of logs where result === FAIL	Every returned log has result === FAIL
T-0 5-0 7	Logs sorted newest first	Call <code>api.getEntryLogs()</code>	Array sorted by scannedAt descending	Each log scannedAt >= the next log scannedAt
T-0 5-0 8	Compliance rate in valid range	Call <code>api.getStats()</code>	Stats object with complianceRate between 0 and 100	0 <= complianceRate <= 100
T-0 5-0 9	Dashboard renders stat cards	Mount Dashboard component	Four stat cards visible after data load	Text Active Workers, Compliance Rate present in DOM
T-0 5-1 0	Workers page renders table	Mount Workers component	Table with at least one worker row	Worker name present in DOM

T-0 5-1 1	Registration modal opens	Click Register Worker button	Modal with form fields visible	Labels Full Name, RFID Card UID, Job Role present in DOM
T-0 5-1 2	Workers page role filter	Mount Workers component	Role filter dropdown visible	Option All Roles present in DOM
T-0 5-1 3	Workers page search input	Mount Workers component	Search input visible	Input with search placeholder present in DOM

5.A.6 Known Limitations and Open Issues

The Admin Panel currently runs against a mock API service because MOD-04 is still under development, so all data displayed in the panel is static and does not reflect real database state. No end-to-end integration test has yet been performed against the live backend. The panel has no authentication mechanism and assumes the local network is trusted, which is consistent with the overall system scope. Worker search is implemented client-side, which is acceptable for the alpha demonstration set of under 50 workers but will degrade as the worker count grows. Pagination on the Entry Logs page uses limit and offset parameters, but the mock service currently returns the full log set, so the pagination behavior cannot be fully verified until the real backend is connected.

Two open issues from the cross-module compatibility review affect the Admin Panel. Issue C-03 concerns the `face_mask` PPE key, which is declared as valid in the Admin Panel interface but has no corresponding detection class in MOD-01. Any role configured with a `face_mask` requirement will cause every inspection of that role to fail permanently until either MOD-01 adds the class to its model or the `face_mask` entry is removed from the PPE catalog. Issue C-04 concerns the shape of the payload that MOD-03 forwards to the Turnstile Display: MOD-03 currently sends a flat worker name string and a string list of PPE items, while the display interface expects a structured worker object and an object-array of PPE items. This must be reconciled before integration testing between MOD-03 and MOD-05 begins.

5.B Turnstile Display

5.B.1 Background and Related Work

Most industrial access control terminals deployed in factories and construction sites communicate inspection results through a single status LED and a buzzer. The team reviewed several such turnstile installations on campus and at the Gebze OSB industrial zone and observed that workers who are denied access have no way of knowing which specific PPE item triggered the rejection. This forces them to walk back, ask a supervisor, and restart the entire scan cycle. This observation directly motivated the choice to build a rich tablet display rather than an LED strip, because the AI module already produces per-item detection results that are wasted unless the display surfaces them to the worker.

For the rendering layer, the team evaluated three alternatives: a plain HTML page driven by a Python tkinter window on the Raspberry Pi itself, a Flutter mobile application, and a React web application served from any HTTP host that the on-site tablet can reach. The plain Python option was rejected because it would tie the display lifecycle to the same process that runs the AI inference, and any UI hang would also stall the gate control loop. Flutter was rejected because the team has prior React experience and because deploying a Flutter web build to a kiosk tablet still ultimately runs in a browser, removing its main advantage. React 19 with Vite was selected because it can be served statically from any LAN host, can be reloaded without rebooting the Raspberry Pi, and supports the team's TypeScript-first workflow.

For the transport layer, the team compared HTTP long-polling, Server-Sent Events, and WebSocket. Long-polling was rejected because the inspection cycle has a hard 10-second budget

and request setup overhead would consume a non-trivial fraction of that budget. Server-Sent Events would have been acceptable for the server-to-display direction but offers no clean reverse channel for the heartbeat frames that MOD-03 needs to confirm the tablet is alive. WebSocket was selected because it provides full-duplex communication over a single TCP connection and is supported natively by every modern browser without any additional library.

5.B.2 Methodology

The application is organized around a single React hook called `useDisplayController` that owns the entire active state of the display. It exposes the current display state, the most recent normalized message, and the connection status. Two collaborator classes feed the hook: `WebSocketClient` is the production transport that implements the published client contract, and `MockSimulator` is the development-only event source used during testing. The top-level App component chooses between the two based on configuration and delegates rendering to seven full-screen components corresponding to the inspection states: `IDLE`, `IDENTIFYING`, `UNKNOWN_CARD`, `INSPECTING`, `GRANTED`, `DENIED`, and `CONNECTION_ERROR`. The rendering layer is kept stateless and easy to snapshot in unit tests, while all timing and lifecycle concerns stay inside the hook.

Data enters the application as a JSON string from the WebSocket message event, passes through a normalization layer for shape validation, and emerges as a strongly-typed discriminated union. The hook stores that message and immediately changes the active display state. After every accepted state change, the hook invokes a callback so the WebSocket client can emit an acknowledgment frame back to MOD-03 confirming that the tablet has rendered the new state. There is no other data path in the module.

Three deliberate design decisions shape the implementation. First, the display does not own the canonical state machine, which lives in MOD-03. The display only mirrors the state that MOD-03 announces, which keeps it testable in isolation and avoids a dual-source-of-truth problem. Second, the display maintains a small local auto-return timer for the three terminal states (`GRANTED`, `DENIED`, `UNKNOWN_CARD`) because the `IDLE` message from MOD-03 may be lost on a flaky link, and a kiosk stuck on a green Access Granted screen would block the worker queue. Third, the WebSocket reconnect logic is bounded to a maximum number of attempts because an indefinite retry loop would mask hardware faults that should be escalated to the duty staff member.

The normalization layer also resolves three documented integration mismatches before data reaches the React layer. The state normalizer accepts both the MOD-03 names `GRANTED` and `DENIED` and the MOD-04 names `PASS` and `FAIL`, folding them into the internal display state enum. The PPE normalizer accepts the list either as a flat string array or as the rich object array declared in the backend interface. The worker normalizer falls back to a flat worker name field when the structured worker object is absent. Frames whose state field cannot be resolved are dropped with a console warning so a malformed payload from MOD-03 cannot crash the kiosk.

5.B.3 Inter-Module Communication

The Admin Panel communicates directly only with MOD-04 Backend. Every page-level action, including worker registration, role PPE reassignment, log filtering, and statistics retrieval, is expressed as a call to one of the REST endpoints listed in the module interface document. This communication boundary exists because MOD-04 owns the single source of truth for all worker and role data, and placing a local database inside the Admin Panel would create two authoritative copies of the same data that would drift out of sync. The direction of dependency is one-way: the Admin Panel depends on MOD-04, but MOD-04 does not depend on the Admin Panel, so the backend can continue serving MOD-03 inspection requests even if the Admin Panel is offline.

The Admin Panel has no direct connection to MOD-03, MOD-01, or MOD-02. This boundary is intentional because the Admin Panel is a management interface, not a real-time control interface, and a direct connection to MOD-03 would place inspection-critical traffic on the same network path as administrative traffic. Any inspection data the Admin Panel needs to display, such as entry log history or compliance statistics, reaches it indirectly: MOD-03 writes entry logs to MOD-04 after each inspection and the Admin Panel reads those logs from MOD-04 on demand. The delay

between a gate event and its appearance in the Admin Panel is equal to however long MOD-03 takes to POST the log, typically under one second, which is acceptable because administrators do not need real-time freshness on historical records.

5.B.4 Implementation

The most non-trivial part of the implementation is the auto-return timer inside `useDisplayController`. The timer is held in a React ref rather than in component state because writing to a ref does not schedule a re-render and the timer ID itself never needs to influence the rendered output. Every accepted message clears any in-flight timer before scheduling a new one, which ensures that a fast sequence such as IDENTIFYING followed immediately by INSPECTING does not leave a stray timeout that would later wipe the screen back to IDLE during the inspection phase. The timeout durations come from the public interface constants, so changing the on-screen dwell time only requires updating the contract file and recompiling.

The WebSocket client is implemented as a plain TypeScript class that owns its own socket instance. Reconnection uses a fixed interval with a bounded attempt counter that resets to zero on every successful connection. An intentional close flag is set before an explicit disconnect call so the close handler can distinguish a deliberate teardown from a network drop, and the client routes every inbound frame through the normalization layer before invoking the consumer callback, which means consumers always receive a valid typed message and never handle raw payloads.

The mock simulator is driven by `setTimeout` rather than by an animation framework because the test suite uses fake timers and needs to advance simulated time deterministically. Mock controls render only when mock mode is enabled through an environment variable or a URL parameter, so a production build cannot accidentally expose the developer-only controls.

5.B.5 Testing Strategy

The Turnstile Display is verified with four Vitest test suites. The state machine suite exercises every transition in the inspection state machine and verifies that the auto-return timers fire after the durations declared in the public interface. The screen rendering suite renders every screen component with React Testing Library and asserts on the user-visible text, protecting against regressions in copy or in the data binding between the controller and the screens. The WebSocket suite replaces the global WebSocket constructor with a mock and confirms the handshake, message dispatch, malformed-frame handling, and reconnection scheduling. A fourth end-to-end suite renders the full application with mock mode enabled and exercises the three demo scenarios from the user-facing controls. All four suites run in under two seconds with `npm run test:run`.

Test ID	Scenario	Input	Expected Output	Pass Criteria
SM-01	Initial state	Hook mounts	State equals IDLE	<code>result.current.state === DisplayState.IDLE</code>
SM-02	IDLE to IDENTIFYING	Receive { state: "IDENTIFYING" }	State changes to IDENTIFYING	State equals <code>DisplayState.IDENTIFYING</code>
SM-03	IDENTIFYING to INSPECTING	Receive INSPECTING with worker and required PPE	State changes to INSPECTING with payload	Worker name and required PPE list match payload
SM-04	IDENTIFYING to UNKNOWN_CARD	Receive { state: "UNKNOWN_CARD" }	State changes to UNKNOWN_CARD	State equals <code>DisplayState.UNKNOWN_CARD</code>

SM-05	INSPECTING to PASS via GRANTED alias	Receive { state: "GRANTED", ... }	State changes to PASS	State equals DisplayState.PASS after normalization
SM-06	INSPECTING to FAIL via DENIED alias	Receive { state: "DENIED", missing_ppe }	State changes to FAIL	State equals DisplayState.FAIL after normalization
SM-07	PASS auto-return	After PASS, advance PASS timeout	State returns to IDLE	State equals DisplayState.IDLE
SM-08	FAIL auto-return	After FAIL, advance FAIL timeout	State returns to IDLE	State equals DisplayState.IDLE
SM-09	Invalid state name	Receive { state: "BOGUS" }	State unchanged with console warning	Previous state preserved, console.warn called
WS-01	DISPLAY_READY on open	Mock socket fires onopen	DISPLAY_READY frame sent	Sent JSON has type === "DISPLAY_READY"
WS-02	Forwards normalized message	Mock socket fires GRANTED with string PPE	Consumer callback called with PASS	Callback receives state === DisplayState.PASS
WS-03	Drops malformed JSON	Mock socket fires non-JSON data	Consumer not called, warning logged	Callback not invoked, console.warn called
WS-04	Reconnect after unexpected close	Mock socket fires onclose after open	Status becomes RECONNECTING	A second WebSocket is constructed after the interval

5.B.6 Known Limitations and Open Issues

The display has no offline fallback. If the Wi-Fi link between the tablet and the Raspberry Pi drops for longer than the maximum reconnect window, the screen latches into a connection error state and stays there until a human intervenes, even if MOD-03 continues to physically operate the gate. The display also assumes the tablet browser is dedicated to this application; nothing prevents an operator from navigating away, and no kiosk-lock mechanism is bundled with the build. Browser autoplay restrictions mean that planned audio cues for the GRANTED and DENIED states are not implemented in this version because they require a user gesture to enable audio on first load.

Three integration issues remain open at the time of writing. Issue C-08 concerns state naming: MOD-03 currently emits GRANTED and DENIED while the display interface uses PASS and FAIL as distinct states; the display normalizes both forms but the team should agree on a single canonical form before deployment. Issue C-03 concerns the face_mask detection class, which MOD-01 does not currently produce; any worker role configured in MOD-04 with a face_mask requirement will permanently fail inspection regardless of what the worker wears. Issue C-04 concerns the PPE payload shape that MOD-03 forwards to the display; MOD-03 currently sends a flat string list while the display interface expects structured objects with icon and display name fields; the display tolerates both shapes through normalization, but icon rendering on the GRANTED and DENIED screens will remain generic until MOD-03 forwards the structured objects produced by MOD-04.